PYTHON PROGRAMMING

Activity Based Learning

10 Case studies:

1. Railway Fare and Reservation Calculator
2. Advanced Numeric Pattern Generator
3. Student Email and Identifier Validation
4. Multi-Stage Examination Eligibility System
5. Sports Tournament Ranking System
6. Food Waste Monitoring System
7. Library Borrowing and Fine Reconciliation
8. Campus Transport Allocation
9. Retail Order Dispatch and Return Analysis
10. University Hostel Room Allocation

1. Case study-6
Railway Fare and Reservation Calculator

Problem Statement:

Calculating fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and specified cancellation penalties.

Objectives:

- To accurately calculate train fares based on travel class per-kilometer rates.
- To apply clear age-based pricing modifiers to determine final ticket costs.
- To evaluate time-sensitive refund deductions when a user cancels a reservation.

Assumptions:

Class Rates (Per Kilometer):

- Sleeper Class (SL): ₹0.50 per km
- AC 3 Tier (3A): ₹1.30 per km
- AC 2 Tier (2A): ₹1.85 per km
- AC 1 Tier (1A): ₹3.10 per km

Age Pricing Modifiers:

- Under 5 years old: Free / 0% of the total base fare.
- Between 5 and 59 years old: 50% of the total base fare.
- 60 years old and above: 40% of the total base fare.

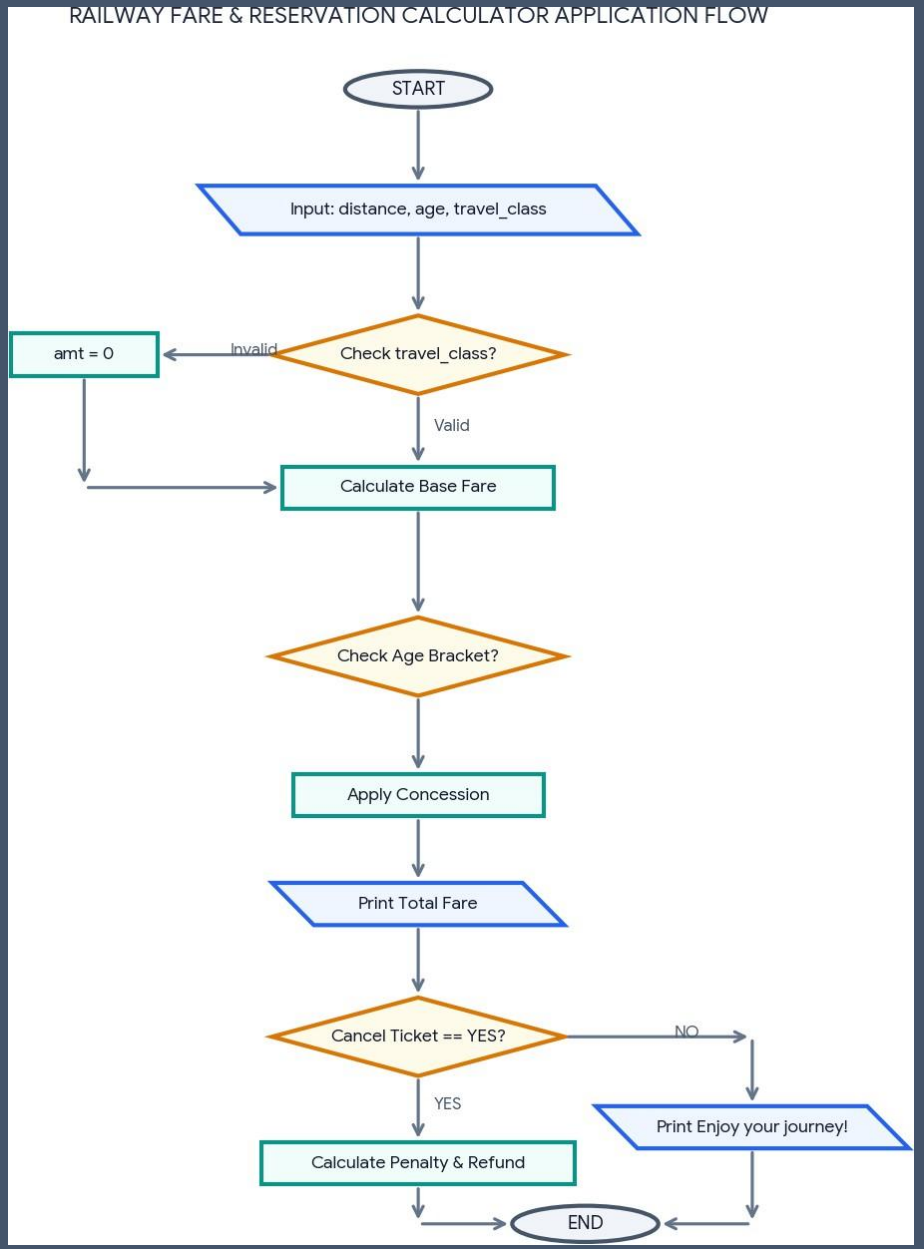
Departure Time-Based Cancellation Penalties:

- ≥ 48 hours remaining: Flat penalty of ₹60.00
- Between 12 and 47 hours remaining: 25% penalty of the calculated ticket fare.
- Between 5 and 11 hours remaining: 50% penalty of the calculated ticket fare.
- Less than 4 hours remaining: 100% penalty (No refund given).

Case study-6

ALGORITHM: RAILWAY FARE & CANCELLATION ENGINE

```
START
  INPUT floating-point value for distance
  INPUT integer value for age
  INPUT string value for travel_class and convert characters to UPPERCASE
  EVALUATE travel_class to calculate base rate (amt):
      - IF travel_class == 'SL' THEN amt = distance * 0.50
      - ELSE IF travel_class == '3A' THEN amt = distance * 1.30
      - ELSE IF travel_class == '2A' THEN amt = distance * 1.85
      - ELSE IF travel_class == '1A' THEN amt = distance * 3.10
      - ELSE PRINT 'Invalid class entered. ' AND amt = 0
  INITIALIZE concession variable c = 0
  EVALUATE age brackets to determine concession coefficient:
      - IF age < 5 THEN c = 0
      - ELSE IF 5 <= age <= 59 THEN c = 50
      - ELSE IF age >= 60 THEN c = 40
  CALCULATE final ticket value: fare = amt * (c / 100)
  PRINT calculated final ticket value formatted as 'Total fare: [fare]' to 2 decimal places
  PRINT string 'Want to cancel the ticket.' to console prompt
  INPUT string for cancel_ticket intent and convert characters to UPPERCASE
  EVALUATE cancel_ticket condition path:
      - IF cancel_ticket == 'YES' THEN:
          * INPUT integer value for time (hours until departure)
          * DETERMINATE penalty based on timeframe thresholds:
              - IF time >= 48 THEN penalty = 60.0
              - ELSE IF 12 <= time < 48 THEN penalty = 0.25 * fare
              - ELSE IF 4 < time < 12 THEN penalty = 0.50 * fare
              - ELSE penalty = fare
          * CALCULATE net balance: refund = fare - penalty
          * PRINT calculated final value as 'Refund amount: [refund]' to 2 decimal places
      - ELSE:
          * PRINT string 'Enjoy your journey!'
  END
```



PSEUDOCODE: RAILWAY FARE CALCULATOR

```
START
    INPUT distance, age, travel_class
    IF travel_class EQUALS 'SL' THEN amt = distance * 0.50
    ELSE IF travel_class EQUALS '3A' THEN amt = distance * 1.30
    ELSE IF travel_class EQUALS '2A' THEN amt = distance * 1.85
    ELSE IF travel_class EQUALS '1A' THEN amt = distance * 3.10
    ELSE PRINT 'Invalid Class' AND amt = 0

    IF age < 5 THEN c = 0
    ELSE IF age BETWEEN 5 AND 59 THEN c = 50
    ELSE IF age >= 60 THEN c = 40

    fare = amt * (c / 100)
    PRINT 'Total fare: ', fare

    INPUT cancel_ticket
    IF cancel_ticket EQUALS 'YES' THEN
        INPUT time
        IF time >= 48 THEN penalty = 60.0
        ELSE IF time BETWEEN 12 AND 47 THEN penalty = 0.25 * fare
        ELSE IF time BETWEEN 4 AND 11 THEN penalty = 0.50 * fare
        ELSE penalty = fare
        refund = fare - penalty
        PRINT 'Refund amount: ', refund
    ELSE
        PRINT 'Enjoy your journey!'
END
```

Case study-6

railway_fare.py

```python
distance = float(input('Enter distance: '))
age = int(input('Enter age: '))
travel_class = input('Enter travel class (SL/3A/2A/1A): ').upper()

if travel_class == 'SL':
    amt = distance * 0.50
elif travel_class == '3A':
    amt = distance * 1.30
elif travel_class == '2A':
    amt = distance * 1.85
elif travel_class == '1A':
    amt = distance * 3.10
else:
    print('Invalid class entered. ')
    amt = 0

c = 0
if age < 5:
    c = 0
elif 5 <= age <= 59:
    c = 50
elif age >= 60:
    c = 40
else:
    pass

fare = amt * (c / 100)
print(f'Total fare: {fare:.2f}')
print('Want to cancel the ticket.')
cancel_ticket = input('Enter Yes or No:').upper()

if cancel_ticket == 'YES':
    time = int(input('Enter hours remaining until train departure:'))
    if time >= 48:
        penalty = 60.0
    elif 12 <= time < 48:
        penalty = 0.25 * fare
    elif 4 < time < 12:
        penalty = 0.50 * fare
    else:
        penalty = fare
    refund = fare - penalty
    print(f'Refund amount: {refund:.2f}')
else:
    print('Enjoy your journey!')
```

Test Cases:

1. Enter distance: 500

Enter age: 35

Enter travel class (SL/3A/2A/1A): 3a

Total fare: 325.00

Want to cancel the ticket.

Enter Yes or No:yes

Enter hours remaining until train departure:7

Refund amount: 162.50

2. Enter distance: 950

Enter age: 12

Enter travel class (SL/3A/2A/1A): sl

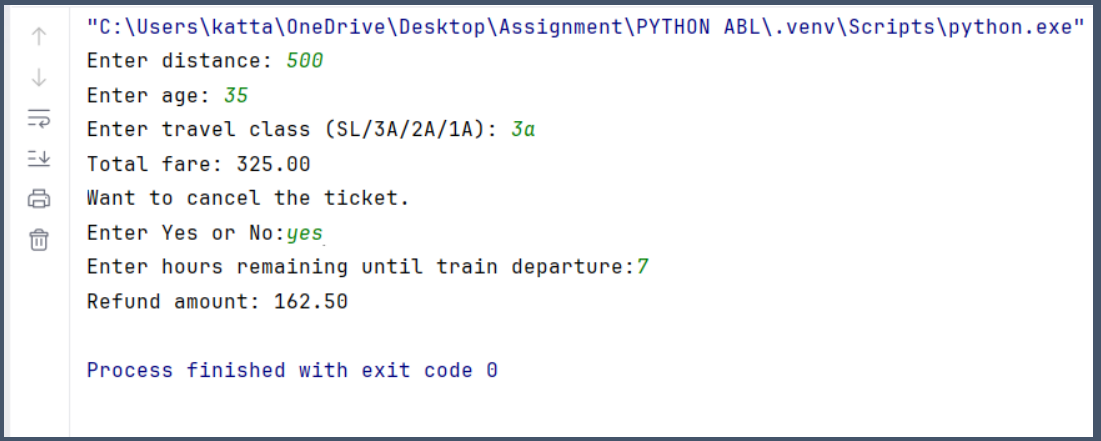
Total fare: 237.50

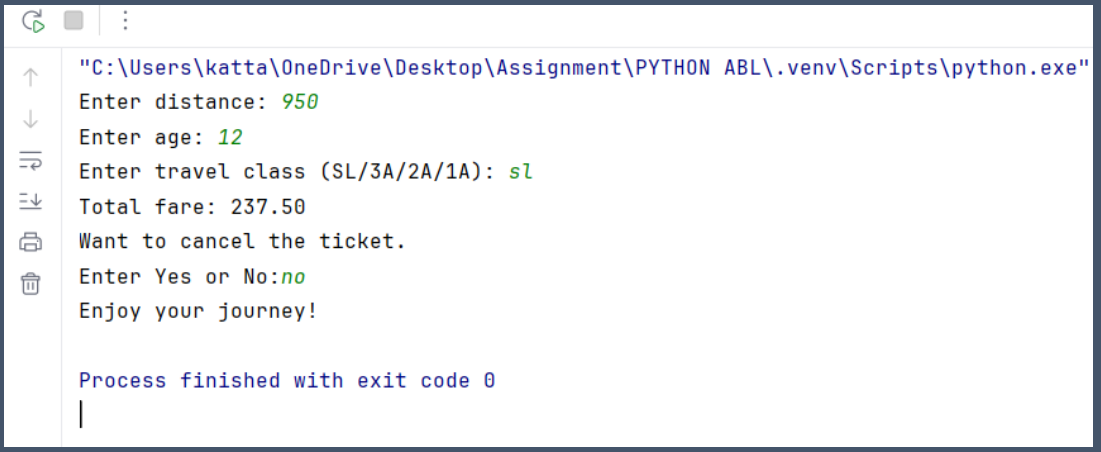
Want to cancel the ticket.

Enter Yes or No:no

Enjoy your journey!

Output:





Conclusion:

The Railway Fare and Reservation Calculator project successfully implements conditional mapping rules to process travel metrics into concrete monetary metrics. By utilizing multi-layered if-elif-else control flows, the program handles travel class fare differences, integrates accurate age-based fee adjustments, and processes conditional dynamic booking cancellation refunds safely based on hours remaining until departure. This assignment highlights how practical logic tables map directly to real-world corporate software features.

2. Case study-27
Advanced Numeric Pattern Generator

Problem Statement:

To produce a hollow diamond or based on user-specified dimensions, with validation for invalid dimensions and carefully aligned output.

Objectives:

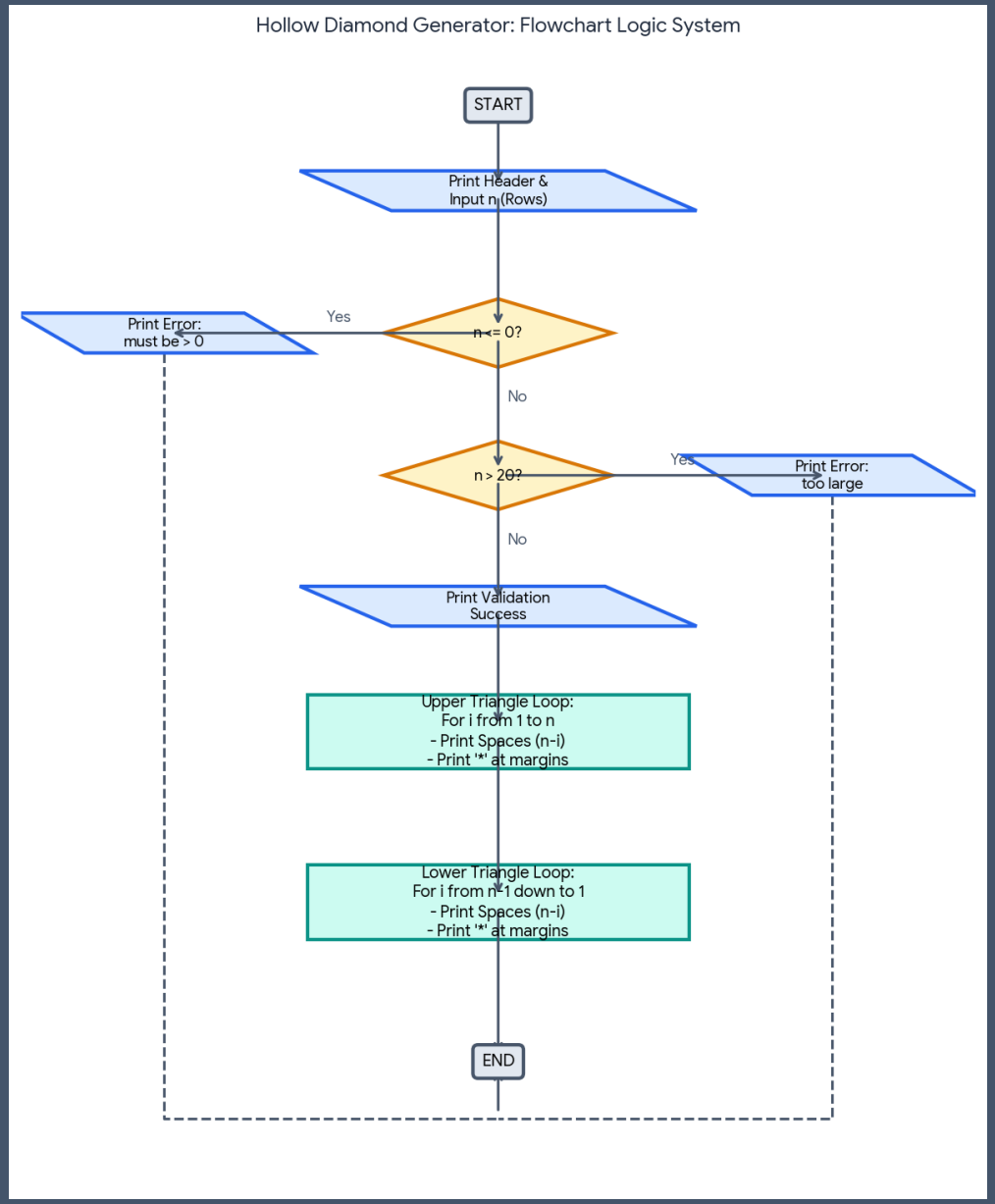
- **Dynamic Grid Layout Rendering:** Programmatically compile a symmetrical hollow diamond structure mapping standard outer edge boundaries via loop configurations.
- **Input Validation Enforcement:** Implement dual-tiered constraint checking gates to cleanly flag structural input violations (n <= 0 or n > 20) prior to execution.
- **Geometric Spacer Optimization:** Coordinate row-indexed space calculation steps (n-i) to preserve sharp spatial alignment during matrix rendering.
- **Conditional Boundary Isolations:** Evaluate character indices dynamically (k == 0 or k == 2*i-2) to strictly restrict character printing to the shape outline.

Assumptions:

- **Single Integer Ingestion:** The console shell expects a single valid integer initialization value mapping directly to the vertical apex midpoint radius (n).
- **Monospaced Display Matrix:** The print pipeline relies on a unified monospaced console font grid context where spaces (" ") match the exact physical width footprint of star elements ("*").
- **Linear Matrix Memory Constraint:** The target device terminal accommodates a horizontal workspace expansion window broad enough to properly host up to 20 row column indexes without causing automatic word-wrapping bugs.

```
ALGORITHM: ADVANCED HOLLOW DIAMOND GENERATOR

1. START
2.   PRINT string 'Hollow Diamond' to indicate script initialization
3.   INPUT integer value for n (number of rows)
4.   EVALUATE dimension constraints via conditional gate validation matrix:
        - IF n <= 0 THEN:
            * PRINT error: 'Invalid Dimension: Number of rows must be greater than 0'
        - ELSE IF n > 20 THEN:
            * PRINT error: 'Invalid Dimension: Number of rows must be less than or equal to 20'
            * PRINT error: 'The number is too larger for screen alignment.'
        - ELSE:
            * PRINT success: 'Dimensions validated! Generating pattern...'
            * EXECUTE UPPER TRIANGLE LOOP CORE (Iterate i from 1 to n):
                - Iterate j from 0 to (n - i - 1): PRINT spaces to align left margins
                - Iterate k from 0 to (2*i - 2):
                    * IF k == 0 OR k == 2*i - 2 THEN PRINT '*' boundary character
                    * ELSE PRINT spacing character ' ' to form the hollow interior
                - PRINT new line break to advance to the subsequent row vector
            * EXECUTE LOWER TRIANGLE LOOP CORE (Iterate i from n - 1 down to 1):
                - Iterate j from 0 to (n - i - 1): PRINT spaces to align left margins
                - Iterate k from 0 to (2*i - 2):
                    * IF k == 0 OR k == 2*i - 2 THEN PRINT '*' boundary character
                    * ELSE PRINT spacing character ' ' to form the hollow interior
                - PRINT new line break to advance to the subsequent row vector
5. END
```



IDE WORKSPACE: PSEUDOCODE_DIAMOND.txt

```
START
   PRINT 'Hollow Diamond'
   INPUT n AS INTEGER

   IF n <= 0 THEN
      PRINT 'Invalid Dimension: Number of rows must be greater than 0'
   ELSE IF n > 20 THEN
      PRINT 'Invalid Dimension: Number of rows must be less than or equal to 20'
      PRINT 'The number is too larger for screen alignment.'
   ELSE
      PRINT 'Dimensions validated! Generating pattern...'

        // RENDER UPPER TRIANGLE SYMMETRY
      FOR i FROM 1 TO n DO
         FOR j FROM 0 TO (n - i - 1) DO
            PRINT ' ' WITHOUT NEWLINE
         END FOR
         FOR k FROM 0 TO (2 * i - 2) DO
            IF k == 0 OR k == (2 * i - 2) THEN
               PRINT '*' WITHOUT NEWLINE
               ELSE
               PRINT ' ' WITHOUT NEWLINE
            END IF
         END FOR
         PRINT NEWLINE
      END FOR

        // RENDER LOWER INVERTED MIRROR
      FOR i FROM (n - 1) DOWNTO 1 DO
         FOR j FROM 0 TO (n - i - 1) DO
            PRINT ' ' WITHOUT NEWLINE
         END FOR
         FOR k FROM 0 TO (2 * i - 2) DO
            IF k == 0 OR k == (2 * i - 2) THEN
               PRINT '*' WITHOUT NEWLINE
               ELSE
               PRINT ' ' WITHOUT NEWLINE
            END IF
         END FOR
         PRINT NEWLINE
      END FOR
   END IF
END
```

Case Study-27

Python Source Code:

```python
print("Hollow Diamond")
n=int(input("Enter number of rows :"))
if n<=0:
    print("Invalid Dimension: Number of rows must be greater than 0")
elif n>20:
    print("Invalid Dimension: Number of rows must be less than or equal to 20")
    print("The number is too larger for screen alignment.")
else:
    print("Dimensions validated! Generating pattern...")
    for i in range(1,n+1):
        for j in range(n-i):
            print(" ",end=" ")
        for k in range(2*i-1):
            if k==0 or k==2*i-2:
                print("*",end=" ")
            else:
                print(" ",end=" ")
        print()
    for i in range(n-1,0,-1):
        for j in range(n-i):
            print(" ",end=" ")
        for k in range(2*i-1):
            if k==0 or k==2*i-2:
                print("*",end=" ")
            else:
                print(" ",end=" ")
        print()
```

Test Cases:

1. Hollow Diamond

Enter number of rows :21

Invalid Dimension: Number of rows must be less than or equal to 20

The number is too larger for screen alignment.

2. Hollow Diamond

Enter number of rows :5

Dimensions validated! Generating pattern...

```
    *
   * *
  *   *
 *     *
*       *
 *     *
  *   *
   * *
    *
```

Output:

```
"C:\Users\katta\OneDrive\Desktop\Assignment\PYTHON ABL\.venv\Scripts\python.exe"
Hollow Diamond
Enter number of rows :8
Dimensions validated! Generating pattern...
              *
            *   *
          *       *
        *           *
      *               *
    *                   *
  *                       *
*                           *
  *                       *
    *                   *
      *               *
        *           *
          *       *
            *   *
              *

Process finished with exit code 0
```

```
"C:\Users\katta\OneDrive\Desktop\Assignment\PYTHON ABL\.venv\Scripts\python.exe"
Hollow Diamond
Enter number of rows :25
Invalid Dimension: Number of rows must be less than or equal to 20
The number is too larger for screen alignment.

Process finished with exit code 0
```

• Conclusion:

- **Robust Structural Stability:** The integration of upfront defensive condition blocks effectively eliminates common runtime formatting issues (such as console clipping or out-of-bounds screen wrapping) before memory-heavy loops start.
- **Flawless Geometric Symmetry:** By coupling nested forward loops with inverted step loops (-1), the coordinate tracking system maps out an exact reflection across the center axis.
- **Optimized Spacing Management:** Relying on the mathematical relationship (n - i) combined with uniform monospaced layout spacers guarantees that the hollow outline retains its proportional diamond shape across any validated dimension value.

3. Case study-30
Student Email and Identifier Validation:

- **Problem Statement:**
- Validate student email-like strings and registration identifiers against supplied formatting rules, reporting malformed separators, invalid character groups, and missing components.

- **Objectives:**
  - **Enforce Domain Constraints:** Restrict registration exclusively to academic institutions by validating the presence of the trailing .edu top-level domain suffix.
  - **Verify Structural Integrity:** Ensure email syntactic compliance by enforcing the strict presence and singular count of the @ delimiter symbol.
  - **Automate Username Extraction:** Programmatically isolate the local-part substring preceding the @ symbol to serve as the user's account name upon validation success.
  - **Proactively Clean User Input:** Sanitize incoming character payloads by stripping outer white space blocks and standardizing to lower case to eliminate duplicate case mismatch errors.

Assumptions:

Prior String Trimming Actions: The script assumes that calling .strip() handles basic outer white spacing, meaning the explicit startswith(" ") conditional clause serves primarily as an absolute catch-all for structural feedback messages.
Single-Phase Local-Part Extraction: The local-part string extracted via .split("@")[0] is assumed to directly represent a clean, human-readable user display name, without needing further special character filtration (e.g., removing dots or underscores).
Synchronous CLI Operational Context: The input collection pipeline is designed for immediate, single-line console string evaluations within a terminal workspace shell environment.

ALGORITHM: INSTITUTIONAL EMAIL REGISTRATION ENGINE

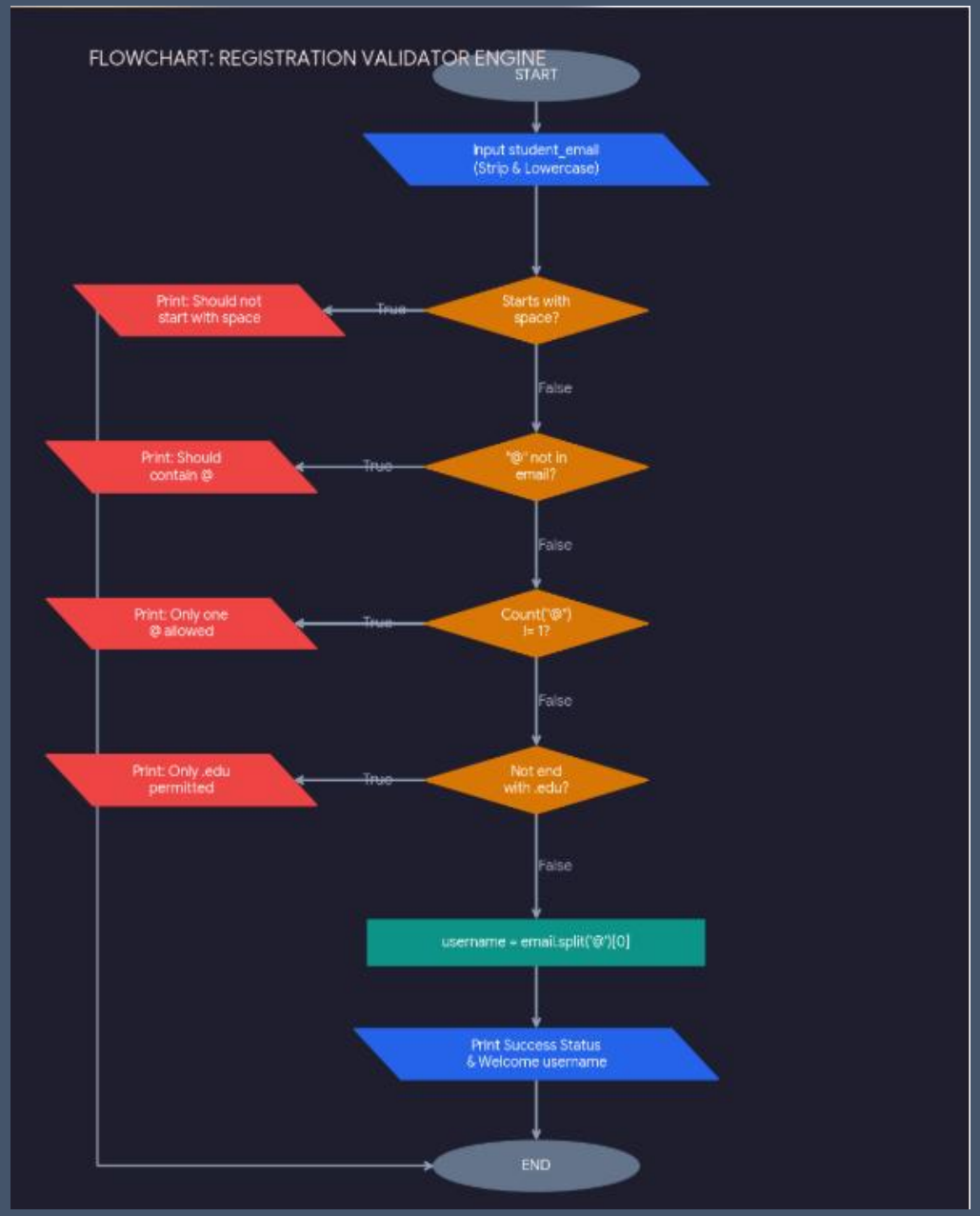
```
1. START

    2. PROMPT user for input and READ string into variable student_email.

    4. EVALUATE structural compliance using a sequential conditional validation matrix:
    - IF student_email starts with a space character ' ' THEN:
    * DISPLAY error message: 'Email should not be started by space.'
    - ELSE IF character '@' is NOT present within student_email THEN:
    * DISPLAY error message: 'Email should contain @ symbol.'

    * DISPLAY error message: 'Email should contain only one @ symbol.'
    - ELSE IF student_email DOES NOT end with the domain substring '.edu' THEN:
    * DISPLAY error message: 'Only institutional .edu emails are permitted.'
    - ELSE (All structural constraints passed successfully):
    * SPLIT the student_email string at the '@' character token boundary.
    * EXTRACT the first substring element (index 0) and assign to variable username.
    * DISPLAY layout header: '----Registration Status : SUCCESS----'

 5. END
```



PSEUDOCODE: INSTITUTIONAL REGISTRATION FILTER

```
START
  INPUT student_email string from console prompt
  SET student_email = student_email.strip().lower()
  IF student_email STARTSWITH ' ' THEN
    PRINT 'Email should not be started by space.'
  ELSE IF '@' NOT IN student_email THEN
    PRINT 'Email should contain @ symbol.'
  ELSE IF COUNT OF '@' IN student_email != 1 THEN
    PRINT 'Email should contain only one @ symbol.'
  ELSE IF student_email DOES NOT ENDWITH '.edu' THEN
    PRINT 'Only institutional .edu emails are permitted.'
  ELSE
    SET username = student_email SPLIT BY '@' at index 0
    PRINT '\n----Registration Status : SUCCESS----'
    PRINT 'Welcome ' + username + '! Your student account has been successfully created.'
  END IF
END
```

Case Study-30

Python Source Code:

```
student_email=input("Enter student email address :").strip().lower()
if student_email.startswith(" "):
    print("Email should not be started by space.")
elif "@" not in student_email:
    print("Email should contain @ symbol.")
elif student_email.count("@")!=1:
    print("Email should contain only one @ symbol.")
elif not student_email.endswith(".edu"):
    print("Only institutional .edu emails are permitted.")
else:
    username=student_email.split("@")[0]
    print("\n----Registration Status : SUCCESS----")
    print(F"Welcome {username}! Your student account has been successfully created.")
```

Test Cases:

1. Enter student email address :kajol123@gmail.com

Only institutional .edu emails are permitted.

2. Enter student email address :jannu45@cse.edu

----Registration Status : SUCCESS----

Welcome jannu45! Your student account has been successfully created.

Output:

```
"C:\Users\katta\OneDrive\Desktop\Assignment\PYTHON ABL\.venv\Scripts\python.exe"
Enter student email address :cse2600@nri.edu

----Registration Status : SUCCESS----
Welcome cse2600! Your student account has been successfully created.

Process finished with exit code 0
```

```
"C:\Users\katta\OneDrive\Desktop\Assignment\PYTHON ABL\.venv\Scripts\python.exe"
Enter student email address :cse2601@nrigmail.com
Only institutional .edu emails are permitted.

Process finished with exit code 0
```

Conclusion:

Robust Input Sanitization: By pairing string trimming (.strip()) with case normalization (.lower()), the engine successfully filters out human input variations, eliminating common duplicate or fragmented credential bugs before structural evaluation begins.

Effective Sequential Validation Sieve: The multi-tiered conditional branching layer serves as an elegant, lightweight firewall. It intercepts syntactic structural defects (like incorrect @ character token volumes) and strict domain compliance requirements (.edu) without needing heavy external regex libraries.

Deterministic Profile Extraction: Upon successful validation, the system clean split orchestration guarantees a predictable, automated extraction of the client's localized username handle, smoothly closing out the registration pipeline with a personalized confirmation layout.

4. Case Study-39
Multi-Stage Examination Eligibility System

Problem Statement:

Determine eligibility for a final examination using attendance, laboratory completion, internal assessment, and fee-clearance conditions, reporting every unmet requirement.

Objectives:

Enforce Tiered Academic Guardrails: Automate the multi-variable screening process required to cross-reference student eligibility metrics prior to processing final exam registrations.

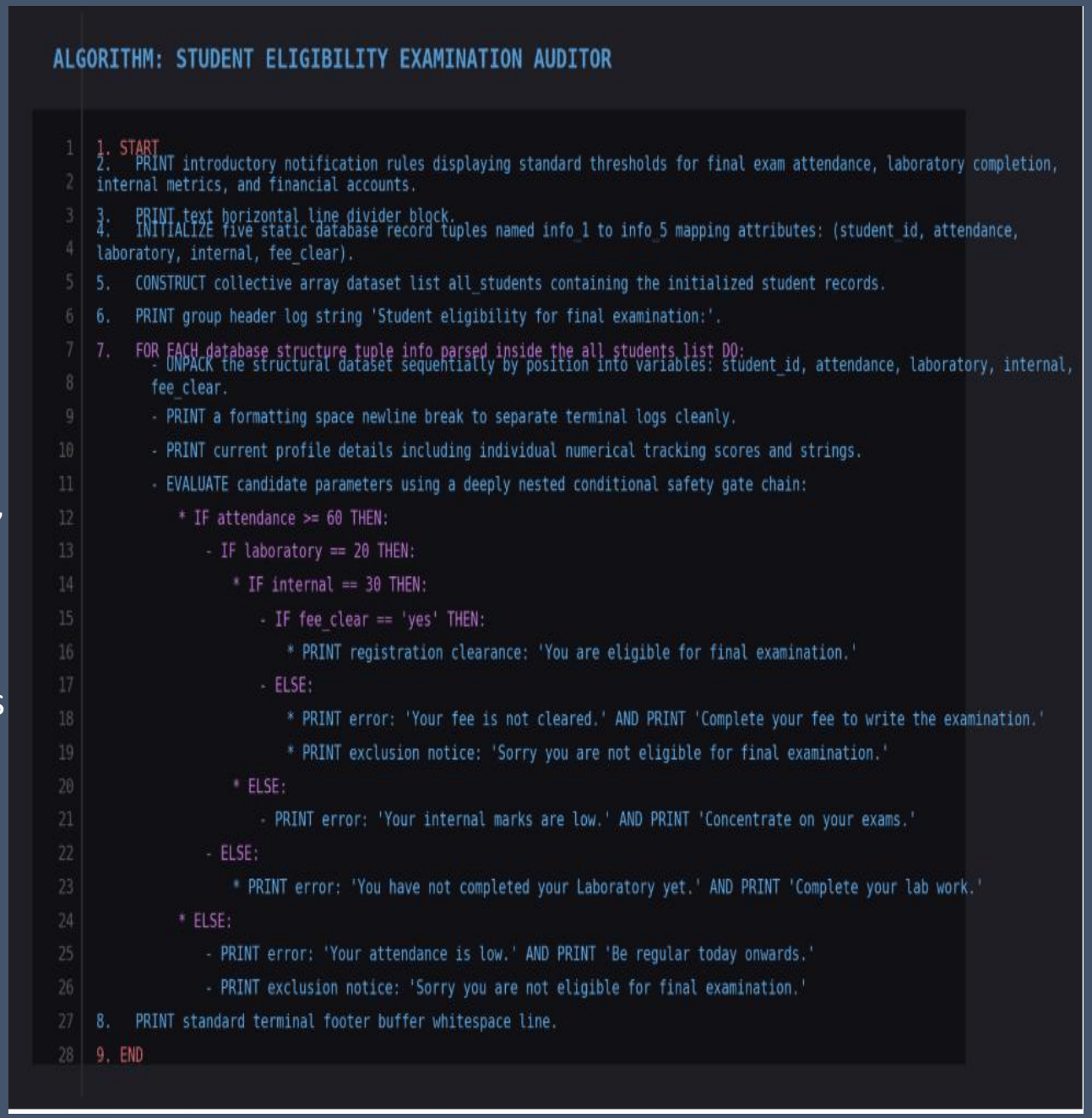
Audit Four Essential Compliance Vectors: Simultaneously evaluate attendance thresholds (\(\ge 60\%\)), strict laboratory completion (exactly \(20\) marks), minimum internal scores (exactly \(30\) marks), and financial clearance ("yes").

Provide Contextual Feedback Logging: Output distinct warning print logs pointing out the exact reason for disqualification (e.g., unpaid balances, low laboratory attendance) if a record triggers an analytical trap step.

Batch Process Data Structures: Map raw records seamlessly inside structured tuple matrix datasets (info_1 to info_5) to trace student metrics cleanly within a sequential database lookup loop

Assumptions:

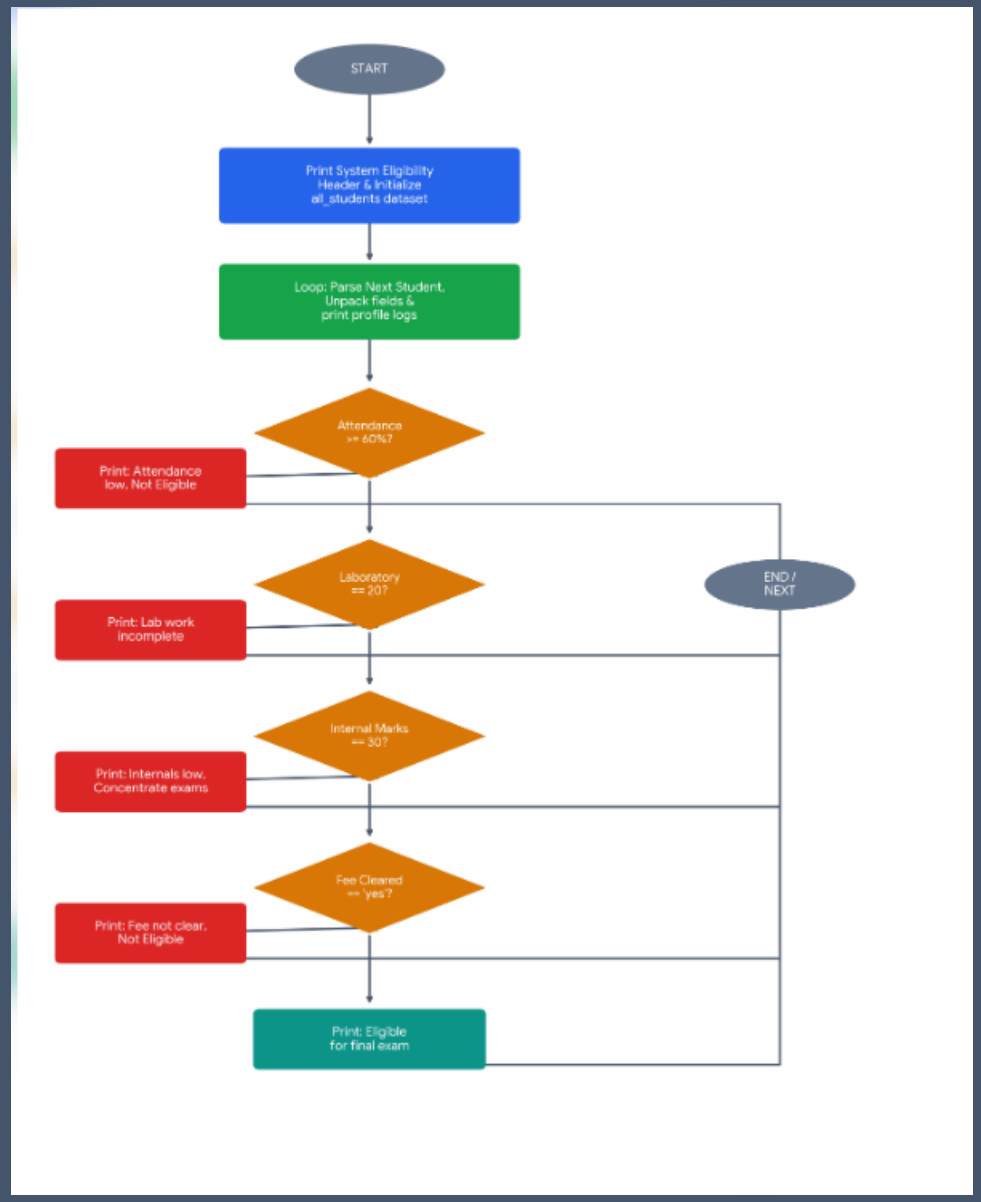
- **Strict Integer/String Literal Typing:** The engine expects specific literal type tracking (like numeric bounds for grades, alongside direct string configurations for financial tracking parameters such as "yes" or "no").
- **Fixed Internal Mark Targeting:** Line 24 assumes that only an exact match (internal == 30) grants pass rights, treating scores either below *or above* 30 as ineligible marks.
- **Static Laboratory Benchmark:** The condition parameter laboratory == 20 assumes that \(20\) marks represents a fixed binary state indicating complete absolute compliance for standard practical courses.



```
ALGORITHM: STUDENT ELIGIBILITY EXAMINATION AUDITOR

1. START
2.   PRINT introductory notification rules displaying standard thresholds for final exam attendance, laboratory completion,
internal metrics, and financial accounts.
3.   PRINT text horizontal line divider block.
4.   INITIALIZE five static database record tuples named info_1 to info_5 mapping attributes: (student_id, attendance,
laboratory, internal, fee_clear).
5.   CONSTRUCT collective array dataset list all_students containing the initialized student records.
6.   PRINT group header log string 'Student eligibility for final examination:'.
7.   FOR EACH database structure tuple info parsed inside the all_students list DO:
       - UNPACK the structural dataset sequentially by position into variables: student_id, attendance, laboratory, internal,
       fee_clear.
       - PRINT a formatting space newline break to separate terminal logs cleanly.
       - PRINT current profile details including individual numerical tracking scores and strings.
       - EVALUATE candidate parameters using a deeply nested conditional safety gate chain:
           * IF attendance >= 60 THEN:
               - IF laboratory == 20 THEN:
                   * IF internal == 30 THEN:
                       - IF fee_clear == 'yes' THEN:
                           * PRINT registration clearance: 'You are eligible for final examination.'
                       - ELSE:
                           * PRINT error: 'Your fee is not cleared.' AND PRINT 'Complete your fee to write the examination.'
                           * PRINT exclusion notice: 'Sorry you are not eligible for final examination.'
                   * ELSE:
                       - PRINT error: 'Your internal marks are low.' AND PRINT 'Concentrate on your exams.'
               - ELSE:
                   * PRINT error: 'You have not completed your Laboratory yet.' AND PRINT 'Complete your lab work.'
           * ELSE:
               - PRINT error: 'Your attendance is low.' AND PRINT 'Be regular today onwards.'
               - PRINT exclusion notice: 'Sorry you are not eligible for final examination.'
8.   PRINT standard terminal footer buffer whitespace line.
9. END
```

Flowchart:



Pseudocode:

```
START:
// Data Definition
DEFINE info_1=(2601,65,15,35,"yes") // LAB INCOMPLETE
DEFINE info_2=(2602,70,20,26,"yes") // LOW INT.
DEFINE info_3=(2603,62,20,30,"no")  // FEE DUE
DEFINE info_4=(2604,60,20,30,"yes") // ELIGIBLE
DEFINE info_5=(2605,50,20,38,"yes") // LOW ATTEND.

SET all_students = [info_1, info_2, info_3, info_4, info_5]

FOR EACH info IN all_students DO
    UNPACK (ID, ATT%, LAB, INT, FEE) FROM info

    DISPLAY: ID, ATT%, LAB, INT, FEE

    IF ATT% >= 60%:
        IF LAB == 20:
            IF INT == 30:
                IF FEE == "yes":
                    OUTPUT: "ELIGIBLE FOR EXAM."
                ELSE:
                    OUTPUT: "FEE DUE: FEE NOT CLEARED."
                    OUTPUT: "Complete fee. NOT ELIGIBLE."
            ELSE:
                OUTPUT: "LOW INT: INT MARKS < 30."
                OUTPUT: "Concentrate. NOT ELIGIBLE."
        ELSE:
            OUTPUT: "LAB INC: LAB NOT COMPLETE."
            OUTPUT: "Complete lab work. NOT ELIGIBLE."
    ELSE:
        OUTPUT: "LOW ATT: ATTEND. < 60%."
        OUTPUT: "Be regular. NOT ELIGIBLE."
    END FOR
END
```

Source Code:

```
print("Important:\n To attend the final examination you should have above 60% of attendance , should complete all laboratory , "
      "should have above 30 marks in internals and fee should be clear. ")
print("-------------------------------------------------------------------------------------------------------------------")
info_1=(2601,65,15,35,"yes")#lab
info_2=(2602,70,20,26,"yes")#internal
info_3=(2603,62,20,30,"no")#fee
info_4=(2604,60,20,30,"yes")#all clear
info_5=(2605,50,20,38,"yes")#attendance

all_students=[info_1,info_2,info_3,info_4,info_5]
print("Student eligibility for final examination:")

for info in all_students:
        student_id, attendance, laboratory, internal, fee_clear =info
        print()
        print("Student ID:",student_id)
        print("Attendance(in percentage):", attendance)
        print("Laboratory(out of 20):",laboratory)
        print("Internal marks(out of 40):", internal)
        print("Fee clear:",fee_clear)
        if attendance>=60:
            if laboratory==20:
                if internal==30:
                    if fee_clear=="yes":
                        print("You are eligible for final examination.")
                    else:
                         print("Your fee is not cleared.");print("Complete your fee to write the examination.")
                         print("Sorry you are not eligible for final examination.")
                else:
                    print("Your internal marks are low.");print("Concentrate on your exams.")
            else:
                print("You have not completed your Laboratory yet.");print("Complete your lab work.")
        else:
             print("Your attendance is low.");print("Be regular today onwards.");
             print("Sorry you are not eligible for final examination.")
print()
```

Output:

```
"C:\Users\katta\OneDrive\Desktop\Assignment\PYTHON ABL\.venv\Scripts\python.exe" "C:\Users\katta\OneDrive\Desktop\Assignment\PYTHON ABL\39.py"
Important:
 To attend the final examination you should have above 60% of attendance , should complete all laboratory , should have above 30 marks in internals and fee should be clear.
--------------------------------------------------------------------------------------------------------------------------------------------------------------------------
Student eligibility for final examination:

Student ID: 2601
Attendance(in percentage): 65
Laboratory(out of 20): 15
Internal marks(out of 40): 35
Fee clear: yes
You have not completed your Laboratory yet.
Complete your lab work.

Student ID: 2602
Attendance(in percentage): 70
Laboratory(out of 20): 20
Internal marks(out of 40): 26
Fee clear: yes
Your internal marks are low.
Concentrate on your exams.

Student ID: 2603
Attendance(in percentage): 62
Laboratory(out of 20): 20
Internal marks(out of 40): 30
Fee clear: no
Your fee is not cleared.
Complete your fee to write the examination.
Sorry you are not eligible for final examination.

Student ID: 2604
Attendance(in percentage): 60
Laboratory(out of 20): 20
Internal marks(out of 40): 30
Fee clear: yes
You are eligible for final examination.

Student ID: 2605
Attendance(in percentage): 50
Laboratory(out of 20): 20
Internal marks(out of 40): 38
Fee clear: yes
Your attendance is low.
Be regular today onwards.
Sorry you are not eligible for final examination.


Process finished with exit code 0
```

5. Case Study-39
Sports Tournament Ranking System

Problem Statement:

Store match results and team scores, calculate total points and tie-break values, and produce rankings under a specified competition policy.

Objectives:

- **Calculate total performance points for each team based on their individual win and tie records.**
- **Determine team rankings dynamically by applying conditional point-range thresholds.**
- **Format and display results clearly, presenting the team name, assigned rank, and total calculated points for each team in the collection.**

Assumptions:

- **Point Values:** A win is strictly worth **50 points**, and a tie is strictly worth **10 points** (derived from the formula win*50 + tie*10). Losses are either not tracked or are worth 0 points.
- **Data Structure:** Each team's record is stored as a tuple containing exactly three elements in a fixed order: (Team Name, Total Wins, Total Ties).
- **Data Types:** The team name is assumed to be a string, while the number of wins and ties are positive integers.
- **Rank Thresholds:** The ranking tier structure operates on fixed boundary limits:
  - **Rank 4:** 150 points or fewer
  - **Rank 3:** 151 to 200 points
  - **Rank 2:** 201 to 300 points
  - **Rank 1:** More than 300 points

Algorithm:

Team Ranking Algorithm

Step 1: Start

Initialize the execution process.

Step 2: Initialize Data

Define four team data structures with (name, wins, ties):

team_1 = ("A", 5, 1)

team_2 = ("B", 3, 1)

team_3 = ("C", 2, 1)

team_4 = ("D", 8, 1)

Store all teams together in a list called teams.

Step 3: Process Each Team (Loop)

For each individual team record inside the teams collection, execute the following sub-steps:

A. Unpack values: Extract name, win, and tie from the current team record.

B. Display identity: Print the header string "TEAM NAME: " followed by the team's name.

C. Calculate points: Compute the value total_pts = (win * 50) + (tie * 10).

D. Evaluate ranking conditions:

IF total_pts is less than or equal to 150: Print "Rank 4".

ELSE IF total_pts is less than or equal to 200: Print "Rank 3".

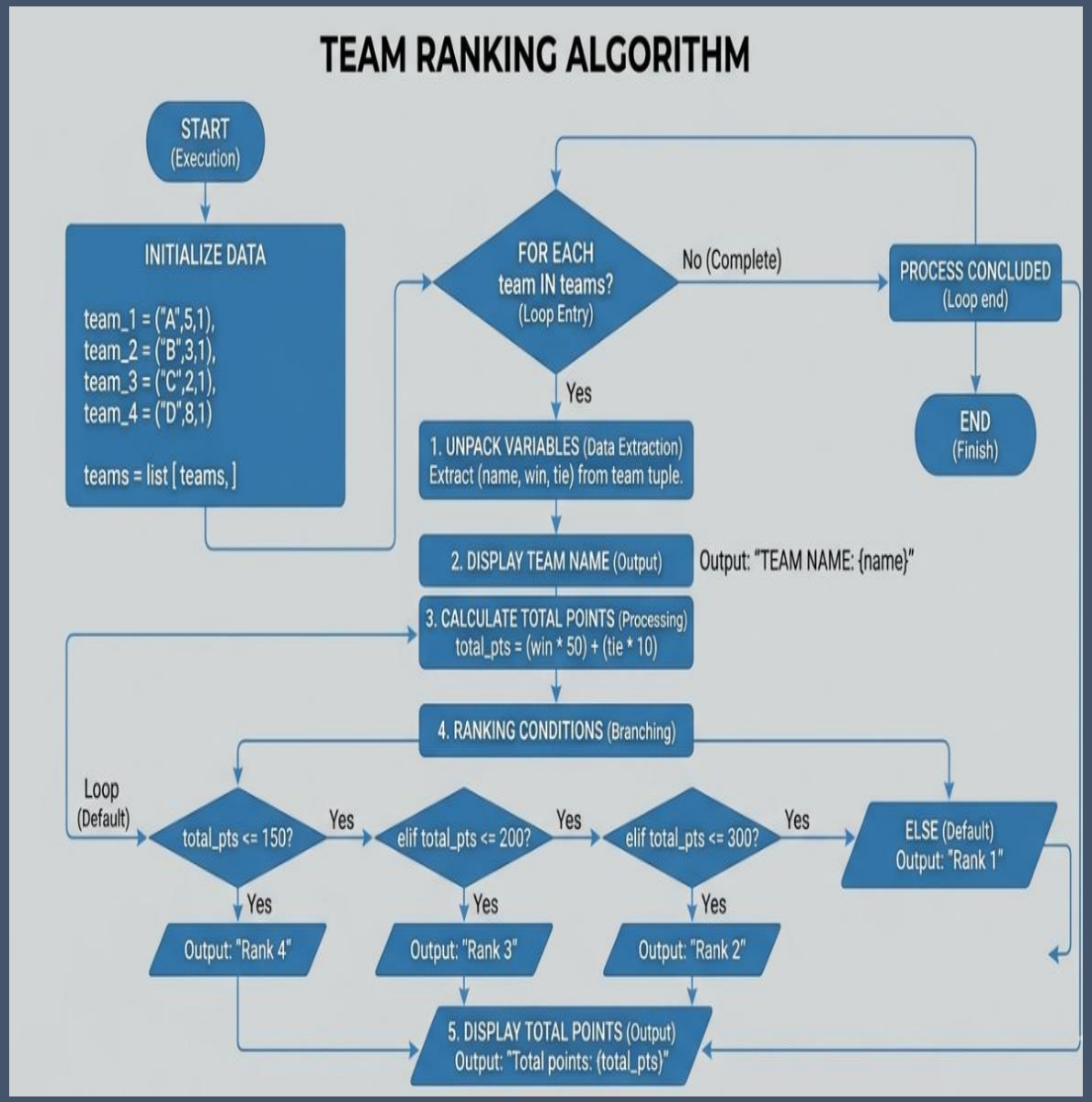
ELSE IF total_pts is less than or equal to 300: Print "Rank 2".

ELSE (points are greater than 300): Print "Rank 1".

E. Display score: Print the string "Total points: "

followed by the calculated total_pts value.

Step 4: End



Pseudocode :

START

// Data Definition

SET team_1 TO ("A", 5, 1)

SET team_2 TO ("B", 3, 1)

SET team_3 TO ("C", 2, 1)

SET team_4 TO ("D", 8, 1)

SET teams TO [team_1, team_2, team_3, team_4]

// Processing Loop FOR EACH team IN teams DO UNPACK name, win, tie FROM team

PRINT "" PRINT "TEAM NAME: " + name

// Perform Math Calculation

SET total_pts TO (win * 50) + (tie * 10)

// Evaluation Logic IF total_pts <= 150 THEN

PRINT "Rank 4" ELSE IF total_pts <= 200 THEN

PRINT "Rank 3" ELSE IF total_pts <= 300 THEN

PRINT "Rank 2" ELSE PRINT "Rank 1" ENDIF PRINT "Total points: " + total_pts ENDFOR

END

Source Code:

```
team_1=("A",5,1)#260
team_2=("B",3,1)#160
team_3=("C",2,1)#110
team_4=("D",8,1)#410
teams=[team_1,team_2,team_3,team_4]
for team in teams:
    name,win,tie=team
    print()
    print(F"TEAM NAME: {name}")
    total_pts=win*50 + tie*10
    if total_pts<=150:
        print("Rank 4")
    elif total_pts<=200:
        print("Rank 3")
    elif total_pts<=300:
        print("Rank 2")
    else:
        print("Rank 1")
    print("Total points:",total_pts)
```

Output:

```
"C:\Users\katta\OneDrive\Desktop\Assignment\PYTHON ABL\.ve
Handball Tournament


TEAM NAME: A
Rank 2
Total points: 260

TEAM NAME: B
Rank 3
Total points: 160

TEAM NAME: C
Rank 4
Total points: 110

TEAM NAME: D
Rank 1
Total points: 410

Process finished with exit code 0
```

6. Case Study-53
Food Waste Monitoring System

Problem Statement:

Compare prepared meals, meals served, and leftovers across several days; calculate wastage percentages and identify days exceeding acceptable limits.

Objectives:

- **Calculate the wastage percentage** for each individual operational day by determining the proportion of leftovers relative to the total prepared food volume.
- **Filter and identify operational anomalies** by running automated conditional checks (if-else) against a predefined threshold (ACCEPTABLE_LIMIT).
- **Generate a clean tabular performance log** in the output terminal to display the tracking status, daily wastage percentages, and compliance evaluation for immediate review.

Assumptions:

- **Constant Variable Shadowing:** The code assumes that reassigning the variable name data within the loop declaration (for day, data in data.items():) will not cause runtime conflicts. *(Note: To avoid potential structural confusion in Python, it is standard practice to rename the loop variant to something like day_data).*
- **Tolerance Boundary:** The acceptable threshold parameter is hardcoded at exactly **15.0%**. Any calculated value strictly greater than 15.0% is flagged as non-compliant.
- **Loss Vector Definition:** The system assumes that all leftovers map entirely to discarded waste metrics. It also assumes that prepared meals are always greater than zero to prevent a potential division-by-zero error (ZeroDivisionError).
- **Data Completeness:** The data structures assume that each daily nested dictionary keyset consistently contains exactly the string literals "prepared" and "leftovers" as integer values.

Algorithm:

Food Waste Monitoring Algorithm

Step 1: Start

Initialize the evaluation program.

Step 2: Initialize Parameters & Data Configurations

Set the tolerance threshold rule: ACCEPTABLE_LIMIT = 15.0.

Define the complete nested data dictionary collection mapping days of the week to their individual dictionary structures (prepared, served, and leftovers).

Step 3: Core Processing Loop (Iterate Days)

For each key-value pair (day, day_info) within the dictionary database, execute the following actions sequentially:

A. Data Extraction: Isolate the nested properties: prepared = day_info["prepared"] and leftovers = day_info["leftovers"].

B. Calculate Performance Metric:

Compute the wastage percentage mapping formula: $\text{waste_percentage}=\left(\frac{\text{leftovers}}{\text{prepared}}\right)\times 100$

C. Evaluate Compliance Constraints:

IF waste_percentage is strictly greater than ACCEPTABLE_LIMIT:

Set status = " EXCEEDS LIMIT".

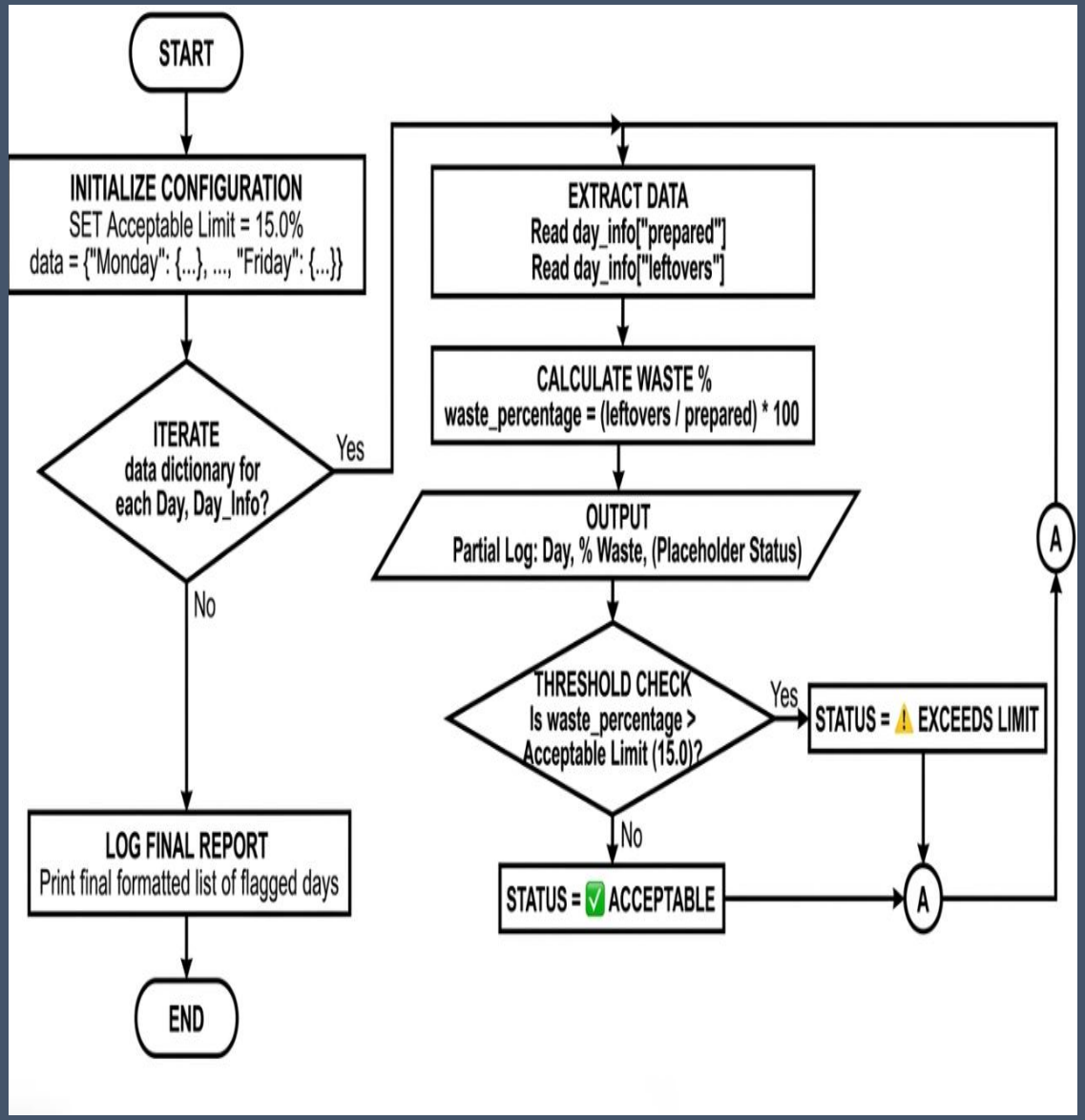
ELSE:

Set status = " ACCEPTABLE".

D. Format Log Output: Print a structured table row showing the current day, the calculated waste_percentage format, and the compliance status.

Step 4: End

Flowchart:



Pseudocode:

START

// 1. Data and Threshold Initialization

SET ACCEPTABLE_LIMIT TO 15.0

SET data TO { "Monday": {"prepared": 500, "served": 420, "leftovers": 80}, "Tuesday": {"prepared": 480, "served": 450, "leftovers": 30}, "Wednesday": {"prepared": 550, "served": 410, "leftovers": 140}, "Thursday": {"prepared": 600, "served": 560, "leftovers": 40}, "Friday": {"prepared": 520, "served": 400, "leftovers": 100} }

PRINT Header formatting: "DAY", "WASTAGE %", "STATUS"

// 2. Loop Through Each Day's Dataset FOR EACH day, day_info IN data DO
// Extract properties from the inner dictionary

SET prepared TO day_info["prepared"]

SET leftovers TO day_info["leftovers"]

// Calculate the percentage of food wasted

SET waste_percentage TO (leftovers / prepared) * 100

// 3. Conditional Threshold Check

IF waste_percentage > ACCEPTABLE_LIMIT THEN SET status TO " EXCEEDS LIMIT" ELSE SET status TO " ACCEPTABLE" ENDIF

// Output current day metrics PRINT day, waste_percentage, status
ENDFOR

END

Source Code:

```python
data={
    "Monday":    {"prepared": 500, "served": 420, "leftovers": 80},
    "Tuesday":   {"prepared": 480, "served": 450, "leftovers": 30},
    "Wednesday": {"prepared": 550, "served": 410, "leftovers": 140},
    "Thursday":  {"prepared": 600, "served": 560, "leftovers": 40},
    "Friday":    {"prepared": 520, "served": 400, "leftovers": 100}
}
ACCEPTABLE_LIMIT = 15.0

print(f"{'DAY':<12} | {'WASTAGE %':<10} | {'STATUS'}")

for day, data_info in data.items():
    prepared = data_info["prepared"]
    leftovers = data_info["leftovers"]

    # Calculate percentage out of total prepared food
    waste_percentage = (leftovers / prepared) * 100
    if waste_percentage > ACCEPTABLE_LIMIT:
        status = " EXCEEDS LIMIT"
    else:
        status = " ACCEPTABLE"

    print(f"{day:<12} | {waste_percentage:>8.1f}% | {status}")
```

Output:

```
"C:\Users\katta\OneDrive\Desktop\Assignment\PYTHON ABL\.venv\
DAY          | WASTAGE %  | STATUS
Monday       |     16.0% |  EXCEEDS LIMIT
Tuesday      |      6.2% |  ACCEPTABLE
Wednesday    |     25.5% |  EXCEEDS LIMIT
Thursday     |      6.7% |  ACCEPTABLE
Friday       |     19.2% |  EXCEEDS LIMIT

Process finished with exit code 0
```

7. Case Study-58
Library Borrowing and Fine Reconciliation

Problem Statement:

Analyse tuples representing book identifiers, borrowing periods, and return delays; calculate fines, identify overdue records, and reconcile book availability.

Objectives:

- **Process collection data sets** by iterating through a list of individual borrowing records stored as tuples.
- **Determine chronological lateness** by dynamically calculating the exact difference between the allowed borrowing duration and the actual time taken before return.
- **Calculate incremental penalty assessments** using a multi-tiered conditional branching (if-elif-else) rate structure.
- **Reconcile physical inventory status** by dynamically logging every completed return record ID into a master availability collection (available_books).
- **Format output updates** to log individual fines alongside an aggregated list of recovered library items.

Assumptions:

- **Tuple Record Consistency:** The program assumes that every item entry in borrowing_records is a tuple containing exactly three elements matching a structural schema layout: (String: Book Identifier, Integer: Allowed Days, Integer: Actual Days).
- **Standard Pricing Tiers:** Late fine tracking assessments operate on hardcoded boundary values:
  - **On-Time/Early Returns (0 days or fewer):** Charges are hardcoded to $0.00.
  - **Short Term Arrears (1 to 5 days late):** The fine is computed at a standard scalar rate of **$0.50 per day**.
  - **Chronic Arrears (6 or more days late):** The penalty calculates a flat foundational tier fee of **$2.50**, plus an elevated surcharge rate of **$1.00 per day** for all days beyond the initial 5-day grace line.
- **Return-to-Stock Logic Strategy:** The system treats the presence of a record within the processed history tracking pool as conclusive evidence that the book has been returned, blindly marking it as back on shelf availability without checking prior catalog records.

Algorithm:

Library Fine Reconciliation Algorithm

Step 1: Start

Initialize the execution process.

Step 2: Initialize Lists and Trackers

Define the database array borrowing_records containing tuple structures for each book: (Book ID, Days Allowed, Actual Days).

Initialize an empty collection array called available_books to trace recovered inventory.

Step 3: Core Processing Loop (Iterate Transactions)

For each individual book dataset inside the borrowing_records list, execute the following sub-steps sequentially:

A. Unpack Variables: Extract the data properties into target names: book_id, allowed_days, and actual_days.

B. Calculate Latency Vector: Compute days_overdue = actual_days - allowed_days.

C. Evaluate Fine Scale Rules (Branching Constraints):

IF days_overdue is less than or equal to 0: Set fine = 0.00 and normalize days_overdue = 0.

ELSE IF days_overdue is less than or equal to 5: Compute fine = days_overdue * 0.50.

ELSE (greater than 5 days late): Compute fine = 2.50 + ((days_overdue - 5) * 1.00).

D. Reconcile Book Stock: Append the target book_id value into the available_books inventory array collection.

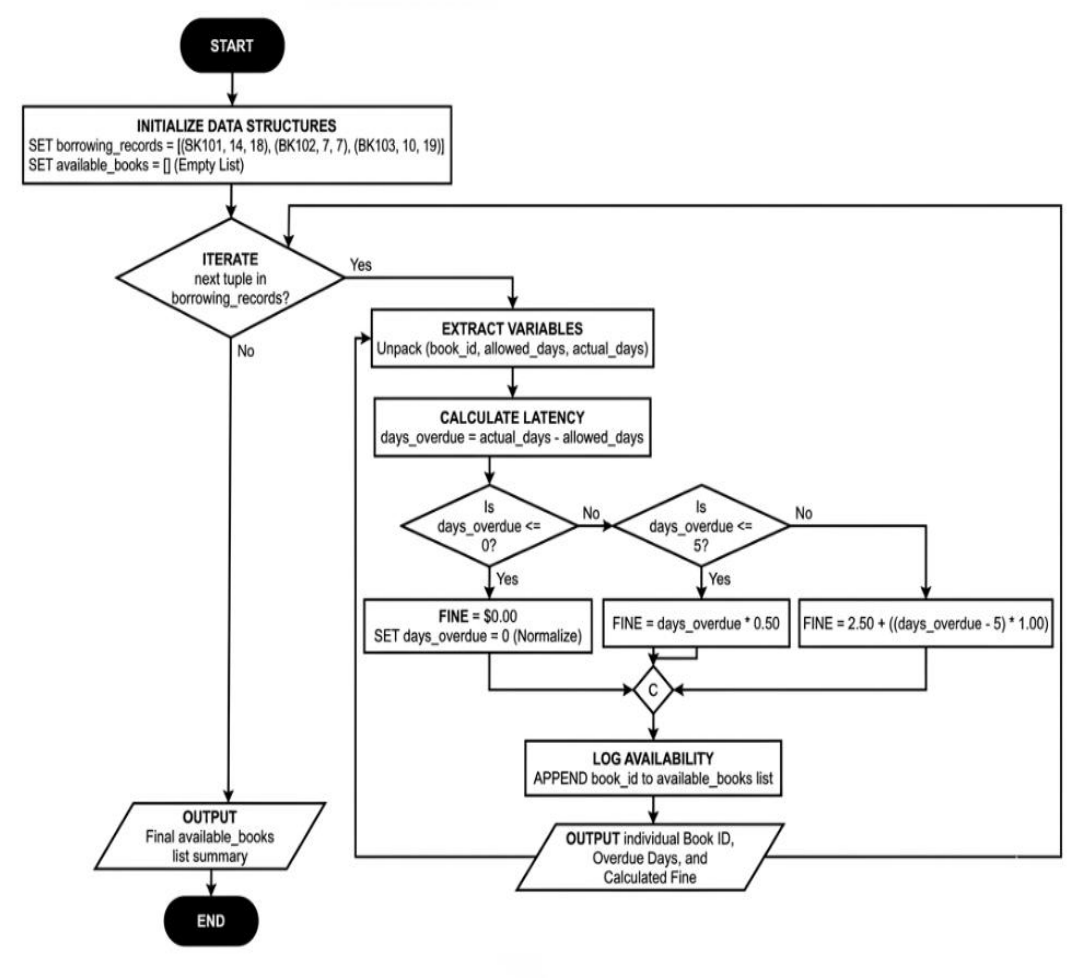
E. Format Terminal Log: Print the values for book_id, days_overdue, and the calculated monetary fine.

Step 4: Generate Executive Summary Report

Output the final consolidated array payload inside the available_books tracking collection list.

Step 5: End

Flowchart:



Pseudocode:

START

// 1. Data and Collections Initialization

SET borrowing_records TO [("BK101", 14, 18), ("BK102", 7, 7), ("BK103", 10, 19)]

SET available_books TO []

// 2. Main Loop: Process Each Book Record FOR EACH book IN borrowing_records DO

// Unpack fields from the current tuple UNPACK book_id, allowed_days, actual_days FROM book // Calculate the number of overdue days

SET days_overdue TO actual_days - allowed_days

// 3. Conditional Fine Logic (Tiered Calculation) IF days_overdue <= 0 THEN

SET fine TO 0.00

SET days_overdue TO 0 ELSE IF days_overdue <= 5 THEN

SET fine TO days_overdue * 0.50 ELSE SET fine TO 2.50 + ((days_overdue - 5) * 1.00) ENDIF

// 4. Reconcile Stock Inventory APPEND book_id TO available_books

// Display individual transaction records PRINT "Book ID: " + book_id PRINT "Days Overdue: " + days_overdue PRINT "Fine Calculated: $" + fine PRINT "" ENDFOR

// 5. Output Final Consolidated Summary Report PRINT "Books Reconciled & Returned to Shelf:" PRINT available_books

END

Source Code:

```python
# Format: (Book ID, Days Allowed, Actual Days Taken)
borrowing_records = [
    ("BK101", 14, 18),  # 4 days late
    ("BK102", 7, 7),    # On time
    ("BK103", 10, 19),  # 9 days late
]

available_books = []

#  Process each book record
for book in borrowing_records:
    book_id, allowed_days, actual_days = book

    days_overdue = actual_days - allowed_days

    if days_overdue <= 0:
        fine = 0.00
        days_overdue = 0
    elif days_overdue <= 5:
        fine = days_overdue * 0.50
    else:
        fine = 2.50 + ((days_overdue - 5) * 1.00)

    # Reconcile availability (Since it was returned, add it back to library)
    available_books.append(book_id)

    # Print individual results
    print(f"Book ID: {book_id}")
    print(f"Days Overdue: {days_overdue}")
    print(f"Fine Calculated: ${fine:.2f}")
    print()


# Print final availability summary list
print("\nBooks Reconciled & Returned to Shelf:")
print(available_books)
```

Output:

```
"C:\Users\katta\OneDrive\Desktop\Assignment\PYTHON ABL\.v
Book ID: BK101
Days Overdue: 4
Fine Calculated: $2.00

Book ID: BK102
Days Overdue: 0
Fine Calculated: $0.00

Book ID: BK103
Days Overdue: 9
Fine Calculated: $6.50



Books Reconciled & Returned to Shelf:
['BK101', 'BK102', 'BK103']

Process finished with exit code 0
```

8. Case Study-59
Campus Transport Allocation

Problem Statement:

Determine the number of buses needed for several routes, account for bus capacities and passenger demand, and identify unserved passengers.

Objectives:

- **Calculate structural transport requirements** for several distinct transit pathways (Ibm, Kondapalli, Bench circle) by parsing a collection dictionary (data).
- **Determine asset utilization limits** by calculating the precise fractional number of vehicles required (demand / bus) and capping deployment using the min() function against available assets (fleet).
- **Quantify service deficits** by computing unmet customer volumes (demand - seats_provided) when local operational bounds limit transportation capacity.
- **Generate a real-time terminal log dashboard** showing demand analytics, deployment details, and unserved passengers for immediate dispatch review.

Assumptions:

- **Constant Bus Capacity Metric:** Every standard shuttle throughout the transit ecosystem shares a rigid, fixed baseline limit of exactly **40 seats** (bus = 40).
- **Floating-Point Vehicle Fractions Allowed:** The logic assumes that vehicle requirements and active deployments can exist as fractional float numbers rather than whole integers (derived from using standard division demand / bus instead of rounding up with integer division). For instance:
  - Ibm calculates a structural need of exactly **1.5 buses**.
  - Bench circle calculates a structural need of exactly **1.75 buses**.
- **Linear Scaling Constraints:** The program assumes that available space perfectly scales linearly alongside the float number of deployments (buses_deployed * bus), meaning a fractional deployment yields a proportional reduction in maximum seated capacity.
- **Perfect Data Layout Completeness:** The script assumes every nested entry inside the main database dictionary consistently matches the exact semantic format keys 'demand' and 'available_fleet'.

Algorithm:

Campus Transport Allocation Algorithm

Step 1: Start

Initialize the fleet distribution calculation process.

Step 2: Initialize Configuration Data

Set the global vehicle tracking parameter: bus = 40 (Capacity per bus).

Define the nested data dictionary data mapping out the routes (Ibm, Kondapalli, Bench circle) along with their individual tracking numbers for 'demand' and 'available_fleet'.

Set the system accumulator tracker: total_unserved = 0.

Step 3: Print Header Interface

Output the clean, left-aligned layout header columns: "ROUTE NAME", "DEMAND", "NEEDED", "DEPLOYED", and "UNSERVED".

Step 4: Process Routing Fleet Metrics (The Loop)

For each individual key-value combination (route, values) inside the tracking data database dictionary, execute these steps in order:

A. Data Extraction: Isolate operational values into variables: demand = values['demand'] and fleet = values['available_fleet'].

B. Calculate Theoretical Asset Need: Compute fractional vehicle demand using standard floating-point division: buses_needed = demand / bus.

C. Evaluate Capped Vehicle Allocation: Restrict deployments to fleet maximums by calculating buses_deployed = min(buses_needed, fleet).

D. Compute Seat Metrics: Determine total available passenger spaces: seats_provided = buses_deployed * bus.

E. Quantify Spillover Passenger Deficits: Compute unserved_passengers = max(0, demand - seats_provided).

F. Update Global Aggregates: Add the current loop results to the tracking master sum: total_unserved += unserved_passengers.

G. Formulate Routing Status Flags:

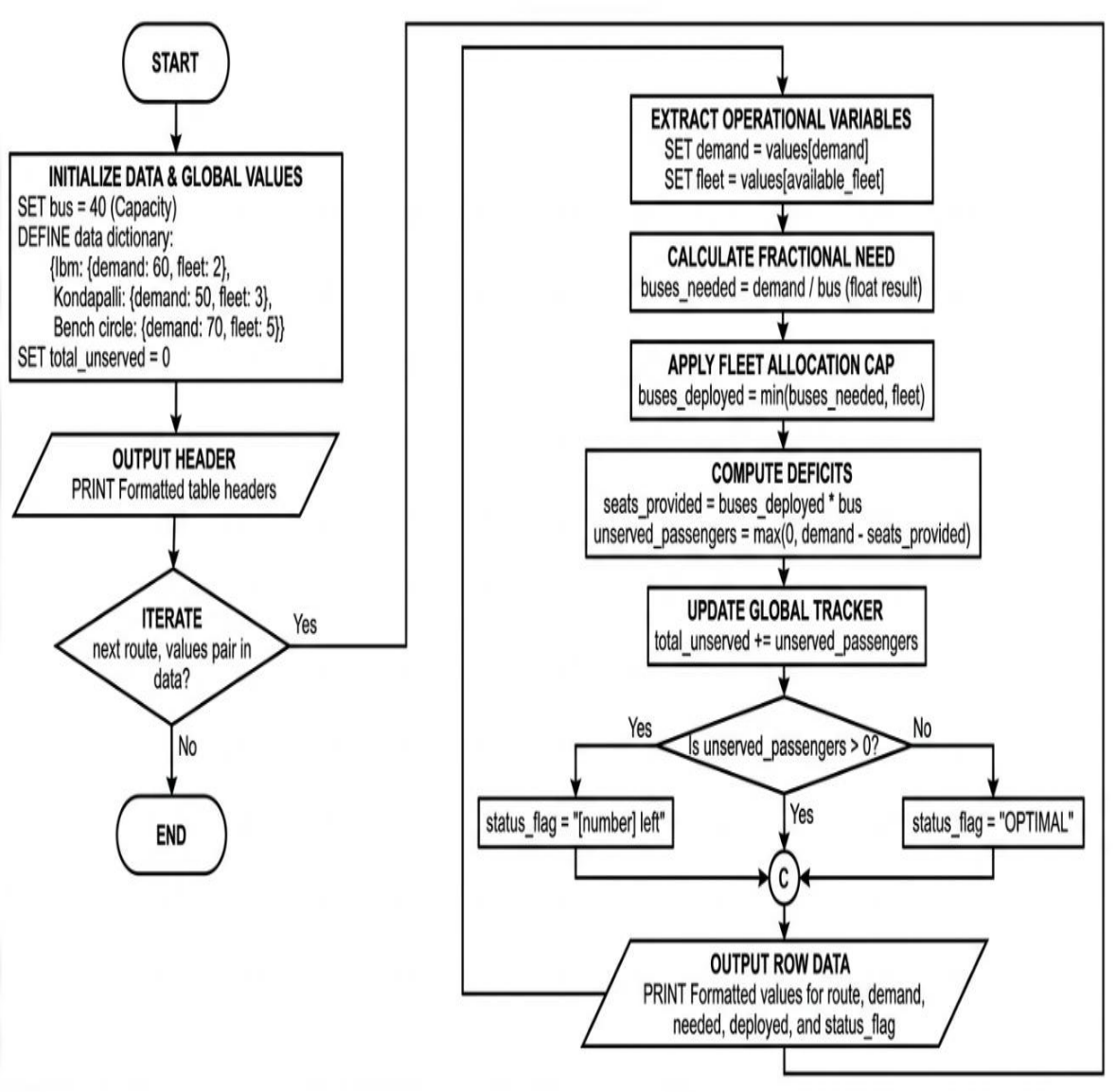
IF unserved_passengers is strictly greater than 0: Set status_flag to read "[number] left".

ELSE: Set status_flag to read "OPTIMAL".

H. Render Row Layout: Print the formatted row metrics showing the current route metrics, float values, and calculated status updates.

Step 5: End

Flowchart:



Pseudocode:

START

// 1. Initialize Configuration & Data Structures SET bus TO 40

// Bus capacity limit SET data TO { 'Ibm': {'demand': 60, 'available_fleet': 2}, 'Kondapalli': {'demand': 50, 'available_fleet': 3}, 'Bench circle': {'demand': 70, 'available_fleet': 5} }

SET total_unserved TO 0 PRINT Table Header layout: "ROUTE NAME", "DEMAND", "NEEDED", "DEPLOYED", "UNSERVED"

// 2. Core Operational Processing Loop FOR EACH route, values IN data DO

// Extract dictionary property metrics

SET demand TO values['demand']

SET fleet TO values['available_fleet']

// Calculate fractional vehicle requirement (float division) SET buses_needed TO demand / bus

// Cap vehicle dispatch based on fleet constraints SET buses_deployed TO MINIMUM(buses_needed, fleet)

// Determine the total passenger slots generated SET seats_provided TO buses_deployed * bus

// Calculate volume deficits/unserved spillover SET unserved_passengers TO MAXIMUM(0, demand - seats_provided)

// Update global aggregate tracker metric SET total_unserved TO total_unserved + unserved_passengers

// 3. Conditional Status Routing IF unserved_passengers > 0 THEN SET status_flag TO unserved_passengers + " left" ELSE SET status_flag TO "OPTIMAL" ENDIF // Print row metrics logging telemetry PRINT route, demand, buses_needed, buses_deployed, status_flag ENDFOREND

Source Code:

```python
bus=40  #bus capacity
data={
    'Ibm':{'demand':60,'available_fleet':2},
    'Kondapalli':{'demand':50,'available_fleet':3},
    'Bench circle':{'demand':70,'available_fleet':5},
}
total_unserved=0
print(f"{'ROUTE NAME':<22} | {'DEMAND':<6} | {'NEEDED':<6} | {'DEPLOYED':<8} | {'UNSERVED'}")
for route,values in data.items():
    demand = values['demand']
    fleet = values['available_fleet']
    buses_needed=demand/bus
    buses_deployed=min(buses_needed,fleet)
    seats_provided = buses_deployed * bus
    unserved_passengers = max(0, demand - seats_provided)
    total_unserved += unserved_passengers
    status_flag = f"{unserved_passengers} left" if unserved_passengers > 0 else "OPTIMAL"
    print(f"{route:<22} | {demand:<6} | {buses_needed:<6} | {buses_deployed:<8} | {status_flag}")
```

Output:

```
"C:\Users\katta\OneDrive\Desktop\Assignment\PYTHON ABL\.venv\Scripts\
ROUTE NAME             | DEMAND | NEEDED | DEPLOYED | UNSERVED
Ibm                    | 60     | 1.5    | 1.5      | OPTIMAL
Kondapalli             | 50     | 1.25   | 1.25     | OPTIMAL
Bench circle           | 70     | 1.75   | 1.75     | OPTIMAL

Process finished with exit code 0
```

9. Case Study-67

Retail Order Dispatch and Return Analysis

Problem Statement:

Process order quantities, dispatch records, and returned quantities; calculate outstanding orders, update inventory, and detect inconsistent records.

Objectives:

- **Process operational inventory workflows** by iterating through raw transaction metrics grouped as a collection of history data records.
- **Quantify supply chain discrepancies** by computing the exact number of outstanding unfilled products (ordered - dispatched).
- **Maintain real-time warehouse counts** by dynamically deducting stock for dispatched units and adding stock back for customer returns.
- **Flag database integrity anomalies** using automated conditional safety checks to filter out impossible operational data logs (e.g., dispatching more goods than ordered).

Assumptions:

- **Data Schema Uniformity:** Every transaction element entry in order_records is a structured tuple matching the layout schema format: (Item Name, Quantity Ordered, Quantity Dispatched, Quantity Returned).
- **Inventory Constraints:** The warehouse is assumed to have enough physical space to receive returned items. It is also assumed that stock names in the order log precisely match the string casing found inside the inventory master dictionary catalog.
- **Single Threat Deficit Boundary:** A data record is classified as "INCONSISTENT" strictly if dispatched > ordered. The logic assumes that other values (like negative numbers or returns greater than dispatches) have already been verified at checkout.

Algorithm:

System Processing Algorithm Text

Step 1: Start - Initialize the warehouse tracking application lifecycle.

Step 2: Initialize Repositories - Define the initial stock level mapping lookup dictionary inventory and populate the transaction list array order_records.

Step 3: Print Visual Grid Header - Output column labels to format standard console displays.

Step 4: Execute Processing Loop (Iterate Records) - For each entry tuple bundle in the collection database, complete these nested actions:

A. Unpack Variables: Isolate data points into target pointers: item, ordered, dispatched, and returned.

B. Run Database Anomaly Validation Check:

IF dispatched is strictly greater than ordered: Set status string flag to " INCONSISTENT" and assign outstanding = 0.

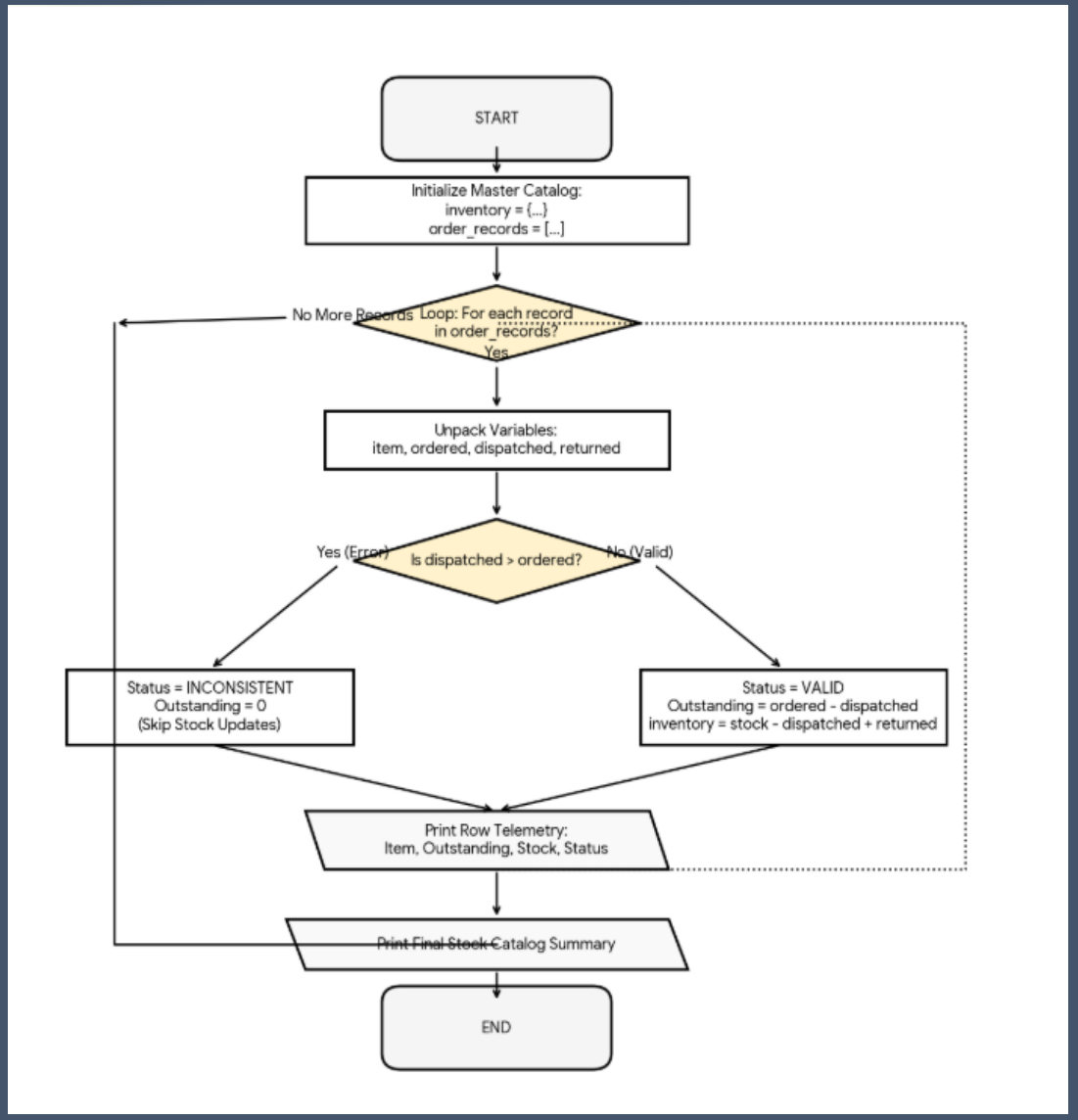
ELSE: Set status string flag to "VALID", calculate outstanding = ordered - dispatched, and update storage levels inside the tracking map array via the mathematical formula: inventory[item] = inventory[item] - dispatched + returned.

C. Format Status Log: Fetch the current stock count and print the formatted row telemetry.

Step 5: Render Final Reports - Output the complete state dictionary showing updated closing inventory counts.

Step 6: End - Terminate program execution.

Flowchart:



Pseudocode:

START

// 1. Storage Configuration Definitions SET inventory TO { "Laptop": 50, "Smartphone": 120, "Headphones": 80, "Tablet": 35 } SET order_records TO [("Laptop", 10, 10, 2), ("Smartphone", 25, 20, 0), ("Headphones", 15, 18, 1), ("Tablet", 5, 5, 0)] PRINT Column Layout Grid: "ITEM", "OUTSTANDING", "NEW STOCK", "RECORD STATUS"

// 2. Core Supply Chain Evaluation Loop FOR EACH record IN order_records DO UNPACK item, ordered, dispatched, returned FROM record

// 3. Logic Branching Validation Engine IF dispatched > ordered THEN SET status TO "INCONSISTENT (Dispatched > Ordered)" SET outstanding TO 0 ELSE SET status TO " VALID" SET outstanding TO ordered - dispatched

// Update inventory numbers if stock profile exists IF item EXISTS IN inventory THEN SET inventory[item] TO inventory[item] - dispatched + returned ENDIF ENDIF SET new_stock TO inventory[item] PRINT item, outstanding, new_stock, status ENDFOR PRINT "Final Updated Inventory Stockroom Count:" PRINT inventory

END

Source Code:

```python
# Format: "Product Name": Current Stock Available
inventory = {
    "Laptop": 50,
    "Smartphone": 120,
    "Headphones": 80,
    "Tablet": 35
}

# Format: (Item Name, Quantity Ordered, Quantity Dispatched, Quantity Returned)
order_records = [
    ("Laptop", 10, 10, 2),      # Normal order, 2 units returned
    ("Smartphone", 25, 20, 0),  # 5 items outstanding (not sent yet)
    ("Headphones", 15, 18, 1),  # INCONSISTENT: Sent more than ordered!
    ("Tablet", 5, 5, 0)         # Normal completed order
]

print(f"{'ITEM':<12} | {'OUTSTANDING':<11} | {'NEW STOCK':<10} | {'RECORD STATUS'}")

for record in order_records:
    item, ordered, dispatched, returned = record

    if dispatched > ordered:
        status = "INCONSISTENT (Dispatched > Ordered)"
        outstanding = 0
    else:
        status = "VALID"
        outstanding = ordered - dispatched
        if item in inventory:
            inventory[item] = inventory[item] - dispatched + returned
    new_stock = inventory.get(item, "N/A")
    print(f"{item:<12} | {outstanding:<11} | {new_stock:<10} | {status}")


print("\nFinal Updated Inventory Stockroom Count:")
print(inventory)
```

Output:

```
"C:\Users\katta\OneDrive\Desktop\Assignment\PYTHON ABL\.venv\Scripts\python.exe"
ITEM         | OUTSTANDING | NEW STOCK  | RECORD STATUS
Laptop       | 0           | 42         | VALID
Smartphone   | 5           | 100        | VALID
Headphones   | 0           | 80         | INCONSISTENT (Dispatched > Ordered)
Tablet       | 0           | 30         | VALID

Final Updated Inventory Stockroom Count:
{'Laptop': 42, 'Smartphone': 100, 'Headphones': 80, 'Tablet': 30}

Process finished with exit code 0
```

10. Case Study-89
University Hostel Room Allocation

Problem Statement:

Allocate students to rooms according to capacity, room eligibility, and specified grouping constraints; detect duplicate assignments and calculate unallocated demand.

Objectives:

- **Automate room allocation categories** based on an input transaction payment tier (fee).
- **Determine custom room dynamics** such as the room number, room type (AC vs. Non-AC), and structural capacity density (room_mates).
- **Calculate comprehensive billing statements** by combining the base tier payment with general food overhead charges (mess_fee) and a custom shared asset rate (maintain_fee).
- **Update institutional capacity records** by dynamically tracking changes to the inventory tracking index (unallocated_room).
- **Log confirmation receipts** out to the console stating the specific apartment metrics, cost details, and current facility occupancy.

Assumptions:

- **Rigid Pricing Structure Configuration:** The matching configuration operates exclusively on four predefined discrete investment thresholds:
    - **Tier A ($50,000):** Triggers a Non-AC room with 8 roommates, a $1,000 base maintenance cost split by room capacity, and room identifier assignment.
    - **Tier B ($70,000):** Triggers a Non-AC room with 6 roommates, a $1,000 base maintenance cost split by room capacity, and room identifier assignment.
    - **Tier C ($100,000):** Triggers an AC room with 4 roommates, a $2,000 base maintenance cost split by room capacity, and room identifier assignment.
    - **Tier D ($200,000):** Triggers an AC room with 3 roommates, a $2,000 base maintenance cost split by room capacity, and room identifier assignment.
- **Fall-Through Exception Logic:** If a user inputs a numeric value that scales less than $70,000 but does not equal exactly $50,000, the application forces an execution route down the else branch of Tier B. Any input value that does not match Tiers A, B, C, or D entirely skips out through the final else: pass boundary without any input error messages.
- **Proportional Inventory Deductions:** The deduction math assumes that allocating a room under a premium tier impacts inventory more heavily than standard tiers (e.g., Tier A decrements by 1 unit, whereas Tier D decrements by 4 units).
- **Flat Mess Fee Overhead:** The cost tracking system assumes a completely flat, non-variable catering charge of exactly **$20,000** applies uniformly across all room assignments regardless of their tier classification.
- **Clean Data Execution:** It is assumed that the interactive operator inputs clear, whole integer values into the command line shell interface to avoid operational data crashes.

Algorithm:

Room Allocation Logic Algorithm

Step 1: Start - Begin the room distribution process.

Step 2: Initialize Constants & Capacity Profiles

Display initial asset baselines: total_room = 110, allocated_room = 70, unallocated_room = 40.

Set general catering charge constant: mess_fee = 20000.

Establish capacity densities per tier: ar = 8, br = 6, cr = 4, dr = 3.

Step 3: Capture Input Payment - Prompt operator to input the payment value into fee.

Step 4: Conditional Allocation Tree (Evaluation Routing)

IF fee <= 70000 (Tier A & B Branch):

IF fee == 50000 (Tier A): Decrement inventory by 1, assign **Room 33**, 8 roommates, Non-AC status, split a $1,000 maintenance fee by 8.

ELSE (Tier B Fall-through): Decrement inventory by 2, assign **Room 61**, 6 roommates, Non-AC status, split a $1,000 maintenance fee by 6.

ELSE IF fee == 100000 (Tier C): Decrement inventory by 3, assign **Room 78**, 4 roommates, AC status, split a $2,000 maintenance fee by 4.

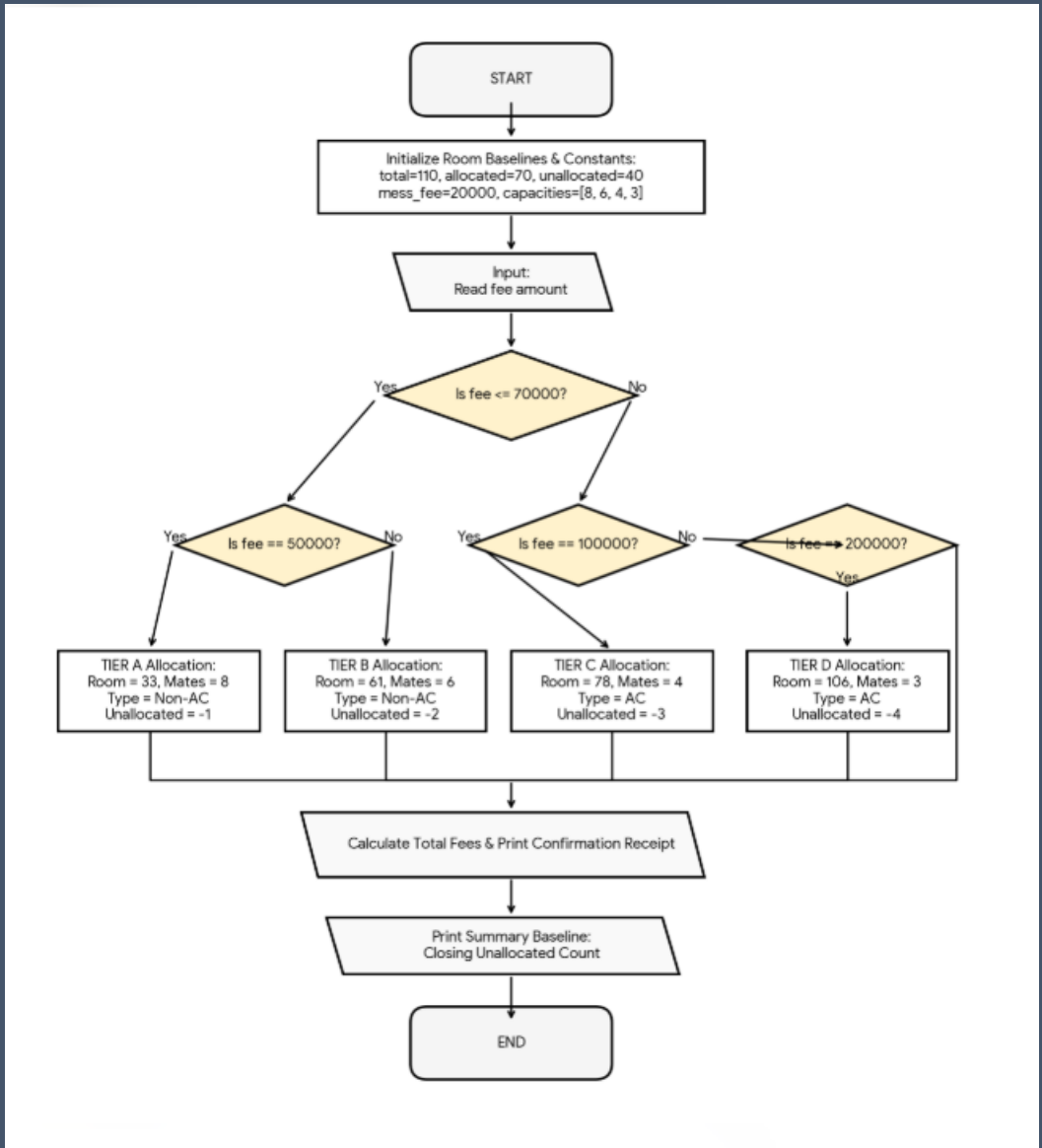
ELSE IF fee == 200000 (Tier D): Decrement inventory by 4, assign **Room 106**, 3 roommates, AC status, split a $2,000 maintenance fee by 3.

ELSE: Skip all operations (pass).

Step 5: Execute Printing Receipts - Calculate total payment overhead sum, display itemized receipts to console, and report final closing unallocated_room metrics.

Step 6: End - Terminate program execution.

Flowchart:



Pseudocode:

START

// 1. Structural Metric Configuration SET total_room TO 110 SET allocated_room TO 70 SET unallocated_room TO 40 PRINT "Total rooms are " + total_room PRINT "Allocated rooms are " + allocated_room PRINT "Unallocated rooms are " + unallocated_room INPUT fee SET mess_fee TO 20000 SET ar TO 8, br TO 6, cr TO 4, dr TO 3

// 2. Room Allocation Routing Tree IF fee <= 70000 THEN IF fee == 50000 THEN SET unallocated_room TO unallocated_room - 1 SET room_num TO 40 - 7 SET room_mates TO ar SET type TO "Non-AC" SET maintain_fee TO 1000

// ar ELSE SET unallocated_room TO unallocated_room - 2 SET room_num TO 70 - 9 SET room_mates TO br SET type TO "Non-AC" SET maintain_fee TO 1000

// br ENDIF SET total TO fee + mess_fee + maintain_fee PRINT "Room Number: ", room_num, " Type: ", type, " Room Mates: ", room_mates PRINT "Total Fee Charged: ", total ELSE IF fee == 100000 THEN SET unallocated_room TO unallocated_room - 3 SET room_num TO 90 - 12 SET room_mates TO cr SET type TO "AC" SET maintain_fee TO 2000

// cr SET total TO fee + mess_fee + maintain_fee PRINT "Room Number: ", room_num, " Type: ", type, " Room Mates: ", room_mates PRINT "Total Fee Charged: ", total ELSE IF fee == 200000 THEN SET unallocated_room TO unallocated_room - 4 SET room_num TO 110 - 4 SET room_mates TO dr SET type TO "AC" SET maintain_fee TO 2000 // dr SET total TO fee + mess_fee + maintain_fee PRINT "Room Number: ", room_num, " Type: ", type, " Room Mates: ", room_mates PRINT "Total Fee Charged: ", total ELSE // Skip invalid entries ENDIF PRINT "At present unallocated rooms are " + unallocated_room

END

Source Code:

```
total_room=110 ;print(F"Total rooms are {total_room}.")
allocated_room=70 ; print(F"Allocated rooms are {allocated_room}.")
unallocated_room=40 ; print(F"Unallocated rooms are {unallocated_room}.")
fee=int(input("Enter your fee(50000,60000,100000,200000):"))
mess_fee=20000 # for all
a=50000 ; ar=8
b=60000 ; br=6
c=100000 ; cr=4
d=200000 ; dr=3
if fee<=b:
    if fee==a:
            unallocated_room=unallocated_room-1
            room_num=40
            room_num=room_num-7
            room_mates=ar
            type="Non-AC"
            mess_fee=20000
            maintain_fee=1000//ar
            total=fee+mess_fee+maintain_fee
            print(F"You are allocated to room number {room_num} , {type} room with {room_mates} room mates and mess fee of {mess_fee}.")
            print(F"Your total fee is {total} including maintenance fee.")
            print("Thank You!")
    else:
            unallocated_room=unallocated_room-2
            room_num=70
            room_num=room_num-9
            room_mates=br
            type="Non-AC"
            mess_fee=20000
            maintain_fee=1000//br
            total=fee+mess_fee+maintain_fee
            print(F"You are allocated to room number {room_num} , {type} room with {room_mates} room mates and mess fee of {mess_fee}.")
            print(F"Your total fee is {total} including maintenance fee.")
            print("Thank You!")
elif fee==c:
        unallocated_room=unallocated_room-3
        room_num=90
        room_num=room_num-12
        room_mates=cr
        type="AC"
        mess_fee=20000
        maintain_fee=2000//cr
        total=fee+mess_fee+maintain_fee
        print(F"You are allocated to room number {room_num} , {type} room with {room_mates} room mates and mess fee of {mess_fee}.")
        print(F"Your total fee is {total} including maintenance fee.")
        print("Thank You!")
elif fee==d:
        unallocated_room=unallocated_room-4
        room_num=110
        room_num=room_num-4
        room_mates=dr
        type="AC"
        mess_fee=20000
        maintain_fee=2000//dr
        total=fee+mess_fee+maintain_fee
        print(F"You are allocated to room number {room_num} , {type} room with {room_mates} room mates and mess fee of {mess_fee}.")
        print(F"Your total fee is {total} including maintenance fee.")
        print("Thank You!")
else:
        pass
print(F"At present unallocated rooms are {unallocated_room}.")
```

Output:

```
"C:\Users\katta\OneDrive\Desktop\Assignment\PYTHON ABL\.venv\Scripts\python.exe" "C:\Users\
Total rooms are 110.
Allocated rooms are 70.
Unallocated rooms are 40.
Enter your fee(50000,60000,100000,200000):60000
You are allocated to room number 61 , Non-AC room with 6 room mates and mess fee of 20000.
Your total fee is 80166 including maintenance fee.
Thank You!
At present unallocated rooms are 38.

Process finished with exit code 0
```

```
"C:\Users\katta\OneDrive\Desktop\Assignment\PYTHON ABL\
Total rooms are 110.
Allocated rooms are 70.
Unallocated rooms are 40.
Enter your fee(50000,60000,100000,200000):70000
At present unallocated rooms are 40.

Process finished with exit code 0
```

Thank You